

ARGUS

A MULTIMODAL GEOSPATIAL INTELLIGENCE PLATFORM

**Predictive urban infrastructure
management through
observe -> understand -> act -> verify**

Final-Year Project Specification

Pilot scope: VIT Bhopal + Kothri

Free / local-first architecture

Primary deep-learning framework: PyTorch

1. Executive Overview

ARGUS is a closed-loop geospatial decision-intelligence platform for urban infrastructure hazards. It combines ground-level citizen evidence, passive civic signals, satellite/environmental context, weather, road-network data and geospatial history to detect, assess, predict, prioritize and verify infrastructure interventions.

The project is designed as a serious final-year engineering system rather than a collection of disconnected AI demos. Every major model produces structured information that feeds the next stage: perception feeds severity, severity feeds risk, risk feeds prioritization, prioritization feeds dispatch, and completed repairs feed verification and the historical learning loop.

The pilot geography is intentionally constrained to VIT Bhopal and Kothri. This keeps data collection, annotation, validation and demonstration tractable while leaving the architecture ready for city-scale expansion.

Core principle

ARGUS should answer three operational questions: What is happening? (descriptive intelligence), what is likely to happen? (predictive intelligence), and what should we do next? (prescriptive intelligence).

ARGUS - CLOSED-LOOP URBAN INTELLIGENCE

1. DATA SOURCES

Citizen images + GPS
Civic text / social signals

2. GEOSPATIAL CORE

PostgreSQL + PostGIS
Spatial de-duplication

3. AI PERCEPTION

PyTorch vision
OpenCV + depth proxy

4. FUSION + RISK

Multimodal confidence
Hazard Priority / ARGUS Score

5. PREDICTIVE ENGINE

7-day risk forecasting
Weather-temporal features

6. PRESCRIPTIVE ENGINE

Crew allocation
Priority-aware routing

7. FIELD + VERIFICATION

Mobile/PWA operations
Before / after verification

8. 3D COMMAND CENTER

Next.js + Cesium
Live hazards + crews

OBSERVE -> UNDERSTAND -> PREDICT -> ACT -> VERIFY -> LEARN

Key design correction: Sentinel-2 is treated as regional/environmental context, not as a direct detector of individual potholes. Ground imagery provides the fine-grained evidence.

2. Problem Statement & Objectives

Urban hazard management is commonly reactive: a defect is noticed, reported, manually triaged, assigned to a crew, repaired and then informally closed. This creates gaps in response time, verification, prioritization and long-term learning.

ARGUS addresses the lifecycle with a unified spatial intelligence layer.

Primary objectives

Objective	ARGUS capability
Multimodal fusion	Combine image, text, GPS, weather, GIS and environmental evidence into a unified hazard record.
Hazard characterization	Detect hazard type and generate a transparent severity score using visual and contextual features.
Predictive maintenance	Estimate 7-day risk using historical hazard observations and environmental/time features.
Closed-loop verification	Compare before/after repair evidence and classify the repair state with confidence.
Passive civic mining	Extract hazard and location signals from public text sources or controlled datasets when live feeds are unavailable.
Dynamic dispatch	Assign and route a limited set of crews against priority-weighted hazards.
Immersive visualization	Expose the operational picture through a 3D digital twin and time-aware dashboard.

Success condition

The project succeeds when a new hazard report can travel through the complete pipeline: ingestion -> geospatial normalization -> AI perception -> risk scoring -> map visualization -> crew dispatch -> repair verification -> audit/history, with measurable evaluation metrics at each major stage.

3. System Architecture

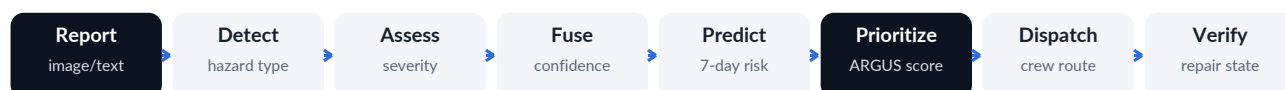
ARGUS is organized as a modular, local-first platform. Heavy model inference is intended to run on the developer machine where practical, reducing dependence on paid AI APIs or cloud GPUs.

Reference architecture

Frontend: Next.js + Tailwind CSS + CesiumJS. Backend: FastAPI. Data: PostgreSQL + PostGIS + object storage.

Workers: Python services for vision, NLP, forecasting, verification and routing. Messaging/cache: Redis.

Deployment: Docker + optional cloud hosting.



Service boundaries

The implementation should separate domain services so individual models can be replaced without rewriting the dashboard. Suggested workers are vision-worker, nlp-worker, forecast-worker, verification-worker and routing-worker. The FastAPI layer becomes the stable contract between the UI, database and intelligence services.

Event-driven flow

NEW_REPORT -> validation -> AI inference -> HAZARD_UPDATED -> risk calculation -> PRIORITY_UPDATED -> optional dispatch -> REPAIR_SUBMITTED -> VERIFICATION_COMPLETED -> case closed / reopened.

4. AI / ML Methodology

PyTorch is the primary deep-learning framework. The project deliberately combines deep learning with classical ML and optimization because the problem contains vision, language, tabular, temporal and combinatorial components.

4.1 Computer Vision

Hazard detection: a PyTorch object-detection or segmentation model identifies potholes, road damage and fallen trees. Severity: derive image features such as affected area, shape, relative depth proxy, road occupancy and visual confidence, then combine them with contextual signals. Important constraint: monocular depth is treated as a relative depth/severity cue unless a reliable scale reference is available; it is not claimed to provide exact centimeter-level pothole depth from a single ordinary RGB image.

4.2 Depth Estimation

MiDaS or Depth Anything can be evaluated as a pretrained depth component. Its role is to produce a depth map that contributes a geometric proxy to severity, not an unsupported absolute physical measurement.

4.3 NLP / Civic Signal Mining

Hugging Face Transformers can support NER and text classification. The pipeline converts unstructured civic text into structured signals: hazard type, location entity, urgency cues, sentiment/context and extraction confidence.

4.4 Predictive Modeling

Build simple baselines first (historical rate / moving average), then compare a tree-based model such as LightGBM or XGBoost with an LSTM only if the available temporal data justifies it. Report MAE/RMSE and calibration/uncertainty rather than claiming guaranteed prediction.

4.5 Repair Verification

Use OpenCV for image normalization and alignment, then compare classical image similarity with a learned embedding/Siamese approach. The output should be Resolved / Partially Resolved / Not Resolved / Uncertain plus a confidence score.

4.6 Routing

Formulate the dispatch problem as a priority-aware vehicle-routing problem. Compare a simple nearest-neighbor baseline, an optimization method such as OR-Tools / genetic algorithm, and an RL policy as the advanced experiment. RL is optional and should be retained only if it can be evaluated fairly against baselines.

5. Multimodal Fusion & ARGUS Risk Score

The strongest architectural feature is not the individual AI models; it is the structured fusion layer that turns heterogeneous evidence into one operational hazard record.

Unified hazard record

Each hazard should carry: hazard ID, type, latitude/longitude, source(s), timestamp(s), detection confidence, severity, recurrence, road context, weather exposure, prediction, estimated response effort, repair state and evidence links.

ARGUS Urban Risk Score

Illustrative formulation: $AURS = w1*Severity + w2*Exposure + w3*Recurrence + w4*WeatherRisk + w5*HistoricalTrend + w6*SourceConfidence$. All terms are normalized to 0-100 and weights are documented rather than hidden in code.

Example explainability panel

Severity +24 Historical recurrence +19 Traffic exposure +17 Weather risk +14 Multiple reports +11 Confidence +6 TOTAL = 91/100

Duplicate / corroboration intelligence

Twenty-seven citizen reports of the same pothole should not become twenty-seven independent tickets. Spatial clustering, temporal proximity and image similarity can collapse corroborating observations into one hazard with multiple evidence sources. The number and reliability of observations then strengthen the confidence of the underlying hazard.

Citizen trust layer

A reporter reliability score can use verified-report rate, false positives, duplicates, location accuracy and historical outcomes. Trust should modify confidence gradually, not dominate a report by itself.

6. Geospatial Intelligence

PostGIS is the central geospatial reasoning layer, not merely a place to store latitude/longitude.

Core operations

Spatial de-duplication; nearest-road matching; hazard-to-road-segment linking; radius queries; hotspot clustering; temporal aggregation; geographic filtering; and map-ready geometry generation.

Multi-resolution strategy

Ground imagery provides fine-grained evidence. Sentinel-2 and other open geospatial layers provide broader environmental context. OpenStreetMap provides road-network structure. Weather contributes time-varying conditions. Terrain/elevation can support the 3D visualization and contextual analysis.

Hotspot example

Cluster #17: 13 observations within a spatial corridor, active for 3 weeks, high severity, elevated weather exposure. ARGUS converts this from isolated complaints into a recurring infrastructure-risk hotspot.

Temporal playback

The dashboard should include a date slider showing hazards appearing, worsening, being repaired and reappearing. This makes the predictive and closed-loop claims visually demonstrable.

7. Zero-Cost / Local-First Strategy

The target is a project that can be developed and demonstrated without paying for commercial AI inference or managed cloud services. Free tiers can be used for convenience, but the core pipeline should continue to function locally.

Layer	Recommended free/local choice	Purpose
Development AI	Antigravity or another coding assistant when available	Code generation, debugging, tests and refactoring
Deep learning	PyTorch	Train/fine-tune vision and verification models locally
Vision utility	OpenCV	Preprocessing, alignment, classical comparison
NLP	Hugging Face Transformers	NER / text classification
Prediction	scikit-learn + XGBoost/LightGBM	Tabular and forecasting baselines
Database	PostgreSQL + PostGIS	Spatial storage and analytics
Cache / jobs	Redis	Queues, caching, event signaling
Maps	OpenStreetMap	Road network and base map context
Satellite / GIS	Sentinel-2 / open GIS data	Regional/environmental context
3D	CesiumJS	Digital twin / terrain / hazard visualization
Packaging	Docker	Reproducible local deployment
Optional inference	ONNX Runtime	Portable optimized model inference

Cost-control rule: use APIs only where they add value. Core perception, scoring, forecasting and routing should not depend on a paid per-request provider.

8. Data Model & API Blueprint

The database should represent the lifecycle, not just the current state.

Core tables

hazards - canonical hazard entity and geometry; observations - individual citizen/social observations; media - image evidence; hazard_scores - severity/confidence/risk outputs; weather_observations - historical/forecast context; road_segments - OSM-derived road graph; predictions - time-indexed forecasts; crews - crew availability/location; dispatch_runs - optimization outputs; repairs - repair action records; verifications - before/after AI results; users and reputation_events; audit_events - immutable lifecycle trail.

Representative API

```
POST /reports
GET /hazards
GET /hazards/{id}
GET /hazards/nearby?lat=&lon;=&radius;=
GET /hotspots
GET /risk/forecast
GET /crews
POST /dispatch/optimize
POST /repairs/{id}/verify
GET /analytics
```

Real-time operations

WebSockets can publish hazard-created, priority-updated, crew-moved, repair-submitted and verification-completed events so the 3D dashboard changes without manual refresh.

9. Evaluation & Research Plan

A strong final-year project must show measurable performance. Every advanced subsystem should have a baseline and a metric.

Subsystem	Baseline	Primary metrics
Hazard detection	Simple / pretrained baseline	Precision, Recall, F1, mAP
Severity	Rule-based score	MAE, RMSE, rank correlation
NER / location extraction	Keyword baseline	Precision, Recall, F1, location accuracy
Forecasting	Moving average / historical rate	MAE, RMSE, MAPE where appropriate
Repair verification	SSIM / classical image difference	Accuracy, Precision, Recall, F1
Routing	Nearest-neighbor	Total distance, response time, priority-weighted coverage
System	Synchronous baseline	API latency, inference time, throughput
Robustness	Clean images	Performance under blur, low light, rain, occlusion, compression

Dataset plan

Create a project-specific ARGUS dataset for the pilot area. Useful annotation fields include hazard class, bounding box/segmentation mask, severity label, GPS, timestamp, road type and before/after pairing. A smaller, carefully annotated dataset is more valuable than an enormous unverified dataset.

Uncertainty

Every model output should expose confidence or uncertainty where practical. The UI should distinguish AI prediction from confirmed field observation.

10. Deliverables & Demonstration

The final demonstration should show the complete operational loop rather than isolated notebooks.

Citizen view

Capture or upload a geotagged hazard image; receive local AI preprocessing; submit the report; see its status and eventual resolution.

Authority command center

3D map of hazards, risk heatmap, predictive zones, crew locations, routes, statistics and an individual hazard intelligence panel.

Field crew view

Assigned route, priority list, navigation context, repair status and after-image submission.

Verification view

Before/after comparison with repair state, confidence and explainable evidence.

Executive analytics

Infrastructure Health Index, active hazards, resolution rate, average response time, route efficiency, prediction performance and recurring hotspots.

What-if simulator

Change number of crews or weather assumptions and show how response time, coverage and predicted risk change.

11. Implementation Roadmap

The project should be built in layers so there is always a working system.

Phase	Build	Exit criterion
Phase 1 - Foundation	Next.js/PWA, FastAPI, PostGIS, report ingestion, Cesium map	A report appears as a geospatial hazard in the dashboard
Phase 2 - Perception	PyTorch detection, image preprocessing, severity scoring	AI result is stored with confidence and shown on the map
Phase 3 - Intelligence	Fusion, de-duplication, hotspots, ARGUS Score	Multiple observations become one ranked hazard
Phase 4 - Prediction	Weather + temporal features, baseline vs tree model vs LSTM experiment	7-day risk forecast with evaluation metrics
Phase 5 - Prescriptive	Crew model, routing baselines, optional RL	3 crews can be assigned to 20+ hazards with measured route quality
Phase 6 - Closed loop	Repair workflow, before/after verification, audit trail	A repair can be verified or reopened through the platform
Phase 7 - Command center	3D polish, temporal slider, analytics, what-if simulation	End-to-end live demonstration is presentation-ready

Scope discipline

Kubernetes, full city-scale deployment, exact physical cost estimation and fully autonomous field robotics are future-scale concepts, not requirements for the final-year prototype. Dockerized modular services, reproducible experiments and a convincing pilot demonstration are the priority.

12. Technical Risks & Mitigations

Risk	Why it matters	Mitigation
Insufficient labeled images	Models may overfit or fail to generalize	Build and annotate a focused pilot dataset; augment and evaluate robustness
Monocular depth scale ambiguity	Exact pothole depth cannot be guaranteed from one RGB image	Use relative depth as a severity feature; label estimates appropriately
Sparse time-series history	LSTM may be unjustified	Use strong classical baselines; only retain LSTM if data volume supports it
Live social-media access	API access, policy or availability may change	Treat passive mining as modular; support replayed/open/synthetic data
RL instability	RL may not outperform deterministic routing	Use OR-Tools / heuristic baselines and make RL an experimental module
Cloud cost	GPU/API services can become expensive	Local-first inference; optional free tiers for deployment
False confidence	AI can make incorrect operational recommendations	Show confidence, evidence and human override

13. Final Project Vision

ARGUS is not a complaint tracker. It is a closed-loop infrastructure intelligence system. The strongest version of the project observes the city from multiple evidence channels, converts those observations into spatially grounded hazards, estimates risk transparently, predicts emerging hotspots, recommends interventions, verifies whether the intervention succeeded, and preserves the lifecycle history for the next decision.

The final demo should make this loop obvious in less than five minutes: report -> detect -> score -> forecast -> dispatch -> repair -> verify -> learn.

Recommended project positioning

Use the title: ARGUS: A Multimodal Geospatial Intelligence and Predictive Decision-Support System for Urban Infrastructure Management. The wording accurately reflects the project's role as an intelligent decision-support platform while leaving room for advanced research components such as RL routing and edge inference.

END OF PROJECT SPECIFICATION

Prepared as the working technical blueprint for ARGUS.